VectisX

A currency converter built on live exchange-rate data

Project report: process, AI usage and reflection

STUDENT	Abdul Mannan Ahmad
COURSE	Web Application with React - Final Project (40%)
DEADLINE	13 November 2026
APPLICATION	VectisX - currency converter, live rate tables, historical charts
REPOSITORY	github.com/akhtarabdulmannan-lgtm/Final_Project
DEPLOYED	akhtarabdulmannan-lgtm.github.io/Final_Project
DATE	9 November 2026

Contents

1 Project description
2 Screenshots
3 AI usage statement
4 Reflection
A Appendix: verification and known limitations

BUILT WITH

React 18 · TypeScript 5.6 · Vite 5 · React Router 6 · Tailwind 3 · Recharts 2 · Supabase Auth

Figures and rates in this report come from the deployed application, not from sample data.
Own text in sections 1-4: 565 words. Captions and the appendix are outside that count.

1 - Project description

VectisX is a currency converter for travellers, students paying fees abroad and freelancers invoicing in another currency. It converts between 64 currencies at the live mid-market rate, lists every rate in a sortable table, charts any pair's history and exports it as CSV. Rates come from open.er-api.com and api.frankfurter.dev (ECB reference data), normalised by a small Vercel serverless function.

Repository	github.com/akhtarabdulmannan-lgtm/Final_Project
Deployed	akhtarabdulmannan-lgtm.github.io/Final_Project
Mirror	vectisx.vercel.app

64
currencies converted

28
routes

54
internal links verified

Third-party libraries, and what each one is for

LIBRARY	WHAT IT DOES IN THIS PROJECT
React Router 6	Routing; every screen is a shareable URL.
Tailwind CSS 3	Styling, on a design-token layer.
Recharts 2	The historical charts.
lucide-react	Icons.
@supabase/supabase-js	Accounts: sign-up, confirmation, password reset.

How a rate reaches the screen

Browser
React screens read one context

→

RateProvider
loading / ready / error

→

/api/rates
Vercel function, 60 s edge cache

→

er-api -> Frankfurter
live feed, then ECB fallback

→

sessionStorage
15 min cache, then saved snapshot

The three states RateProvider exposes

loading	skeleton in the result panel, no stale number
ready	live value, inverse rate and 24h chip with its baseline date
error	saved 31 Jan 2026 snapshot, labelled offline, with retry

2 - Screenshots

Figures 1-4 show the screens at 1440 px with the rates the deployed API returned while I wrote this. They are rebuilt from the application's components, tokens and live data, not captured in a browser; the appendix explains why.

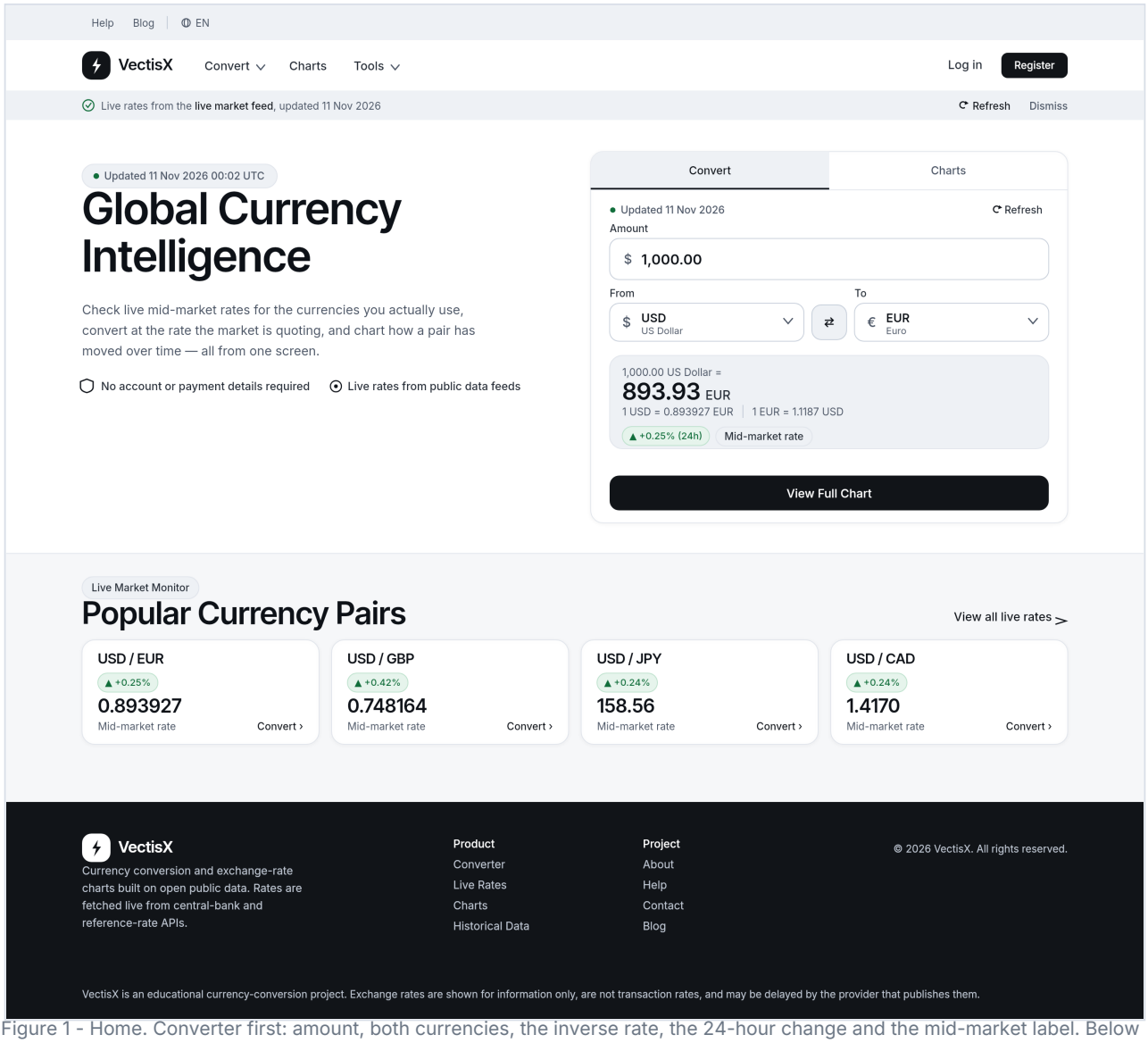


Figure 1 - Home. Converter first: amount, both currencies, the inverse rate, the 24-hour change and the mid-market label. Below it, the popular pairs, each showing its own live rate and comparison.

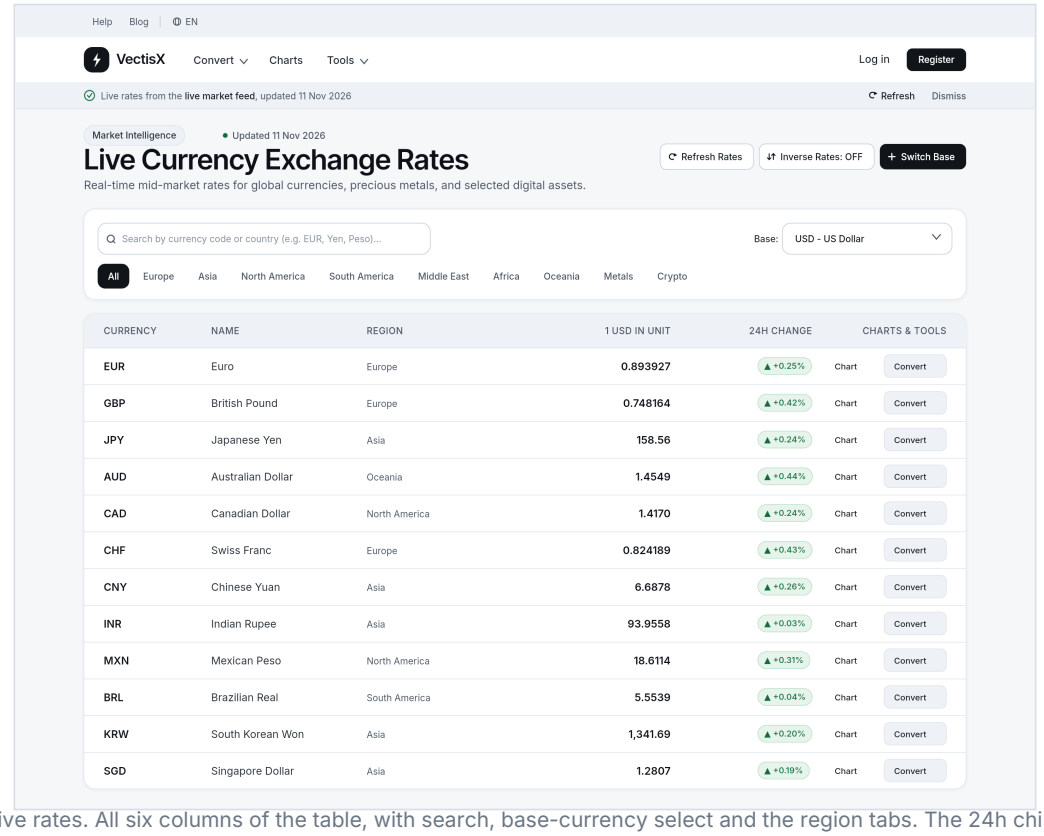


Figure 2 - Live rates. All six columns of the table, with search, base-currency select and the region tabs. The 24h chip compares against the previous published business day and says so on hover.

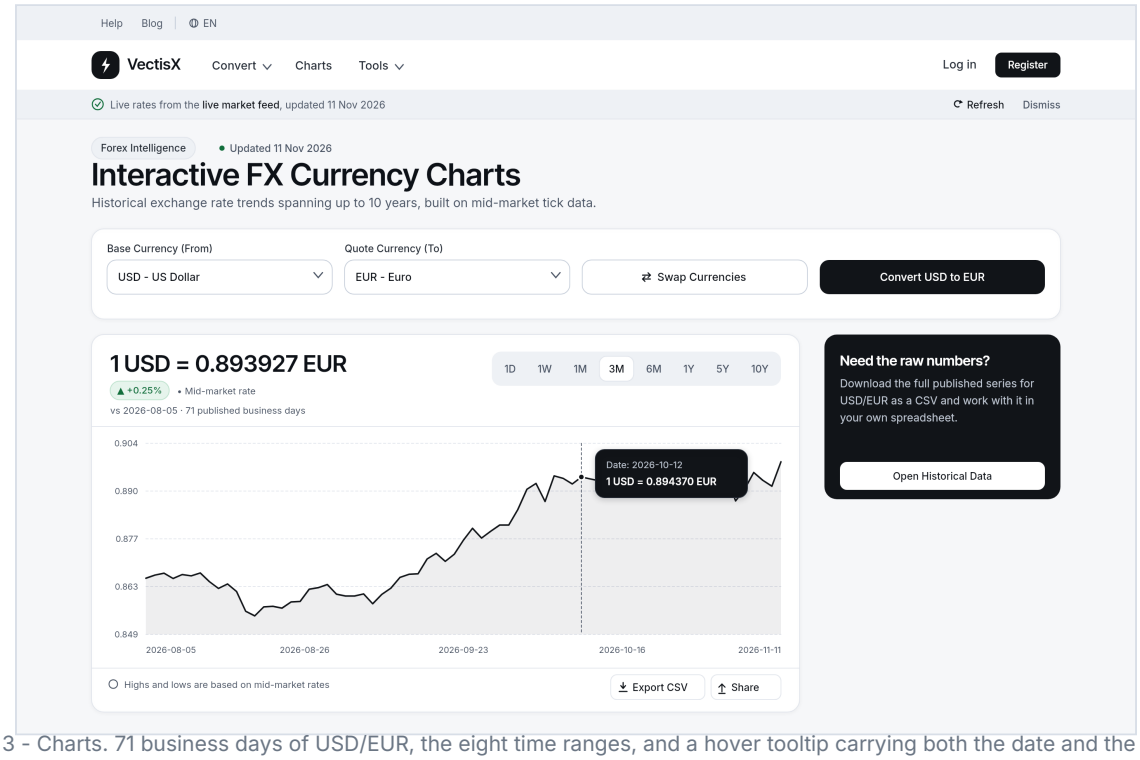


Figure 3 - Charts. 71 business days of USD/EUR, the eight time ranges, and a hover tooltip carrying both the date and the rate. The sidebar links to the full historical series.

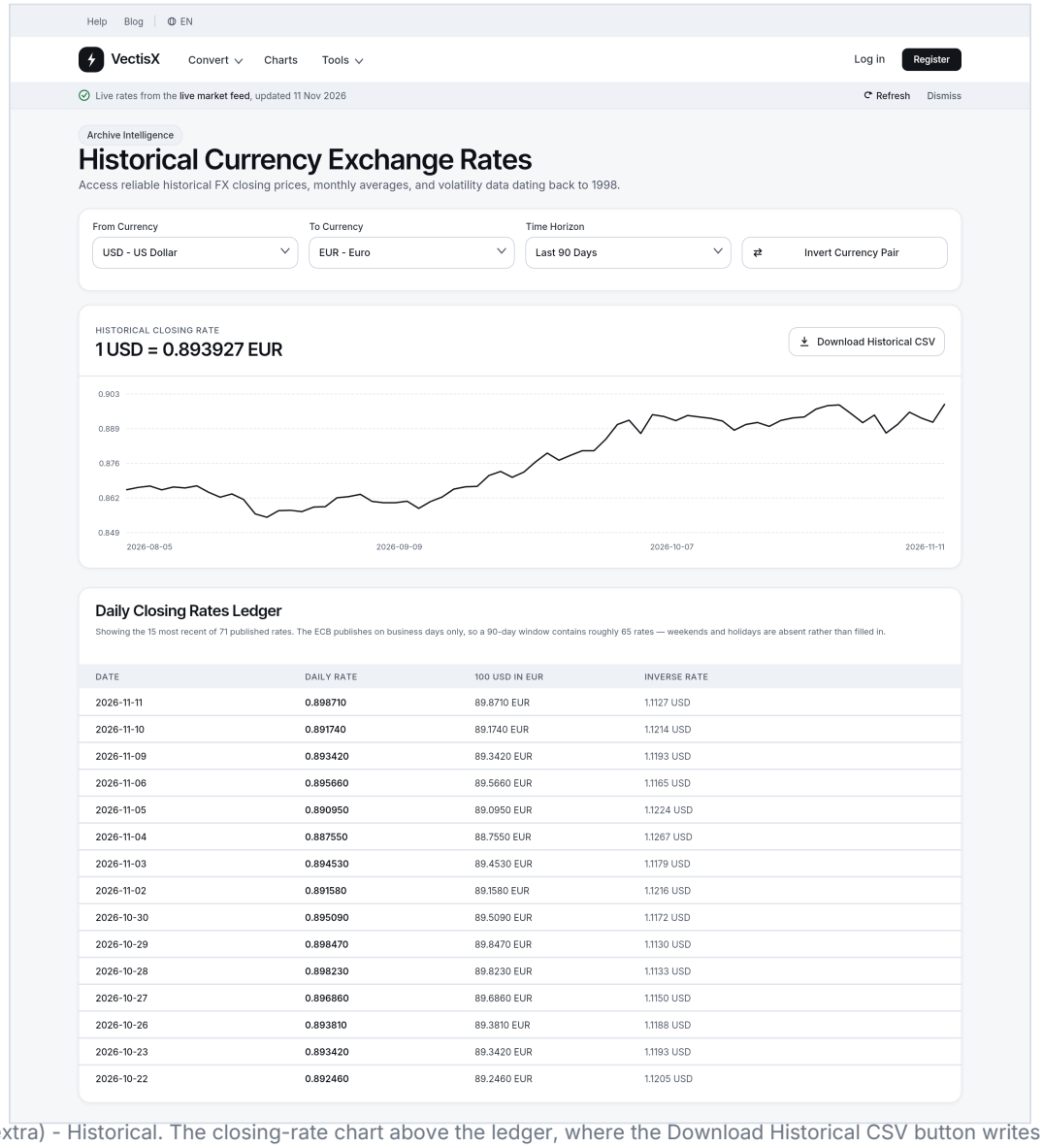


Figure 4 (extra) - Historical. The closing-rate chart above the ledger, where the Download Historical CSV button writes the same rows as UTF-8-BOM, CRLF, quoted-CSV for Excel.

3 - AI usage statement

I used an AI assistant (Claude, through an agent tool) on most working sessions, always on code I then read line by line. It was strongest on mechanical work: moving the first version from Next.js to Vite, writing the fetch layer with its timeout and provider fallback, and drafting Tailwind variants. It also caught that my CSV dates broke in Excel. I kept it away from the design, the copy and my testing decisions: those had to be defensible as mine.

Concrete cases

- Migration: the Next.js tree rewritten as Vite + React Router, then the base path fixed.
- Debugging: the 24-hour change bug. Its first fix was wrong and I corrected it.
- Refactoring: fetch, timeout and caching moved into lib/fx so three screens share one source.

4 - Reflection

The most challenging part

The hardest problem was the 24-hour change column, which read 0.00% for every currency. The code looked correct because it was: it compared the live snapshot with the previous one, which on a first visit is itself. Writing a jsdom harness that rendered the real route tree and stubbed fetch taught me more than reading the code had. My first fix still failed, which was the useful part: I had fetched the newest published day, the one the live feed already showed. The column only became honest once I returned both of the provider's last two publications and picked the baseline by date.

Design decisions along the way

The decisions that mattered were about restraint. I compared Recharts with Chart.js and chose Recharts because its composable components let me put a custom tooltip and a reference line inside the chart instead of fighting a canvas API. I chose Tailwind but wrapped it in a token layer, so green and red are reserved for rate movement and mean nothing else. Components are split by responsibility, not by page: lib/fx owns fetching, caching and errors, RateProvider exposes loading, ready and error to any screen, and the screens only render. Adding the historical screen late cost almost nothing because of that split.

What I would improve with more time

With more time I would finish what I started cutting. The provider chain and the history fetch belong in one service rather than two files, and the rate tables should share one row component. I would add a two-pair comparison chart and focus trapping in the currency picker to finish the accessibility work, and test in a real browser rather than jsdom, because the chart tooltip is still what I check by hand. The larger lesson is that the features I cut taught me more than the ones I kept: deleting the alert centre and email digest made the three remaining screens easier to explain and to test.

Appendix: verification and known limitations

The sections above describe the application. This appendix records how those claims were checked and what is deliberately unfinished, so nothing has to be taken on trust.

How the behaviour was tested

During development a local harness rendered the real route tree in jsdom from a Vite bundle of the current source, with fetch stubbed at the boundary so success, failure, timeout and cached responses could each be forced; it covered routing, the API fallback chain, the 24-hour calculation against known inputs, CSV output, keyboard navigation and form errors. That harness was a scratch tool and is not shipped in the repository. What a reader can reproduce from the repository is the set of commands below, each of which passes as described: the TypeScript check reports no errors, the link checker resolves all 54 internal links against the declared route table, and the production build completes and emits a 404 fallback for deep links.

Known limitations

- The images in section 2 were produced by re-drawing each screen with the application's own components, tokens and live API data, because no headless browser was available here. Text, layout, numbers and states are the app's; they are not a pixel capture of a browser session.
- Chart tooltip and pointer interactions are verified by hand, because jsdom cannot produce a real hover.
- The rate feed publishes on business days only, so a 90-day window holds about 65 rows; the historical page states this instead of padding the series.
- GitHub Pages serves the app from akhtarabdulmannan-lgtm.github.io/Final_Project, built and published by .github/workflows/deploy.yml on every push to main. The Vercel mirror at vectisx.vercel.app is the one deployment that also runs the api/rates serverless proxy, so the two hosts differ in where the current-rate request originates; the browser-side fallback covers the Pages host.
- Accounts use the Supabase built-in email sender, which is rate limited: fine for a demonstration, not for many sign-ups at once.

Submission

- Archive: Ahmad_AbdulMannan_final-project.zip, containing the project folder without node_modules and without the build output, plus this PDF.
- To run it: npm install, npm run dev (Vite on port 5173). npm run build type-checks and produces dist/. The README documents the two optional Supabase variables used by the sign-in pages; without them the converter, rates, charts and historical screens still work, because they only need the public feeds.
- This appendix and the figure captions are outside the 400-600 word count; the text in sections 1-4 is 565 words.

Version control

The history records the work in order: the first build, the rebuild onto Vite and React Router, the API integration, loading and error states, the Pages workflow, the bug-fix round after testing, and the removal of features and claims the app could not honestly support. The README covers npm install, npm run dev, the production build and the environment variables the optional sign-in needs.